

# Timer Systemd

David Rodera

## DEFINICIÓN Y ESTRUCTURA

En systemd, un timer es una unidad de configuración que funciona como un **despertador** para automatizar tareas. Como no realiza la tarea por sí mismo, **siempre está vinculado a una unidad de servicio (.service)**. En esta relación, el timer define el "**cuándo**" y el servicio el "**qué**", permitiendo que el sistema ejecute procesos.

Los archivos de configuración tienen una extensión **.timer**, pero ese mismo archivo debe ir acompañado de un archivo con extensión **.service**. La ruta para archivos con **sudo** es **/etc/systemd/system/**.

### SINTAXIS ARCHIVO .timer

```
[Unit]
Description=Descripción clara de
cuándo se ejecuta esta tarea

[Timer]
# Elige una o varias directivas de
tiempo

# Si se llama igual que el timer, no
es estrictamente necesario.
Unit=mi-tarea.service

[Install]
WantedBy=timers.target
```

### SINTAXIS ARCHIVO .service

```
[Unit]
Description=Descripción clara
After=network-online.target
Wants=network-online.target

[Service]
Type=oneshot
# oneshot hará que el script se
ejecute
User=tu-usuario
ExecStart=/usr/bin/bash
/ruta/absoluta/a/mi-script.sh

[Install]
WantedBy=multi-user.target
```

Posteriormente, se deberá de activar el timer con **sudo systemctl enable --now mi-tarea.timer**

## DIRECTIVAS DE TIEMPO

- **OnCalendar:** Define ejecuciones en fechas y horas precisas, la sintaxis de la directiva de tiempo es **DíaSemana Año-Mes-Día Hora:Minuto:Segundo**.
- **OnUnitActiveSec:** Define un temporizador relativo, desde que se activa la última activación del servicio asociado, por ejemplo: **OnUnitActiveSec=2h**.
- **OnBootSec:** Activa el servicio tras transcurrir un tiempo determinado desde que el sistema arrancó, por ejemplo: **OnBootSec=5min**.
- **Persistent:** Comprueba si el servicio debería haberse ejecutado mientras el equipo estaba apagado. Si es así (**Persistent=true**), lo ejecuta inmediatamente al encenderlo.

## COMPARATIVA

| Aspecto                    | Cron                                                     | Systemd Timers                                                                            |
|----------------------------|----------------------------------------------------------|-------------------------------------------------------------------------------------------|
| Sintaxis y Legibilidad     | <b>Menos legible</b> pero más compacta                   | Más verbosa pero <b>más legible</b>                                                       |
| Integración y Dependencias | <b>Aislado</b> , es ciego ante fallos de red o servicios | <b>Total</b> , espera a que la red suba o que otros servicios finalicen antes de arrancar |
| Tareas perdidas            | <b>No integrado</b> , requiere instalar anacron          | <b>Integrado</b> mediante la directiva Persistent                                         |
| Control de Recursos        | <b>Limitado</b> , según lo que permite el procesador     | <b>Excelente</b> , al permitir usar cgroups                                               |
| Depuración y Estado        | <b>Difícil</b> , rebuscando en /var/log/syslog           | <b>Muy fácil</b> , con systemctl status y journalctl                                      |

Systemd integra anacron o at en como funciones:

- Incluye las funciones de anacron en [Timer], con las funciones de Persistent, que ejecuta tareas pendientes tras un apagado, y RandomizedDelaySec, que evita la saturación al arranque escalonando su inicio aleatoriamente.
- Para simular at, se usa la directiva OnCalendar con una fecha exacta (sin repetición).

## CASO PRÁCTICO

- **Objetivo:** Crear un script simple (ej: gitpull.sh) que actualice un repositorio de GitHub (ej: Grado-Superior-ASIR) con el fin de llevar al día los archivos.
- **Paso 1:** Una vez creado el script, crearemos el service unit (ej: git-pull.service), que ejecutará nuestro script.
- **Paso 2:** Crearemos el timer unit (ej: git-pull.timer), que definirá que el script se ejecute 1 minuto después iniciar el equipo, y que se vaya ejecutando cada minuto.
- **Paso 3:** Activar y arrancar el timer.
- **Paso 4:** Verificar que está activo y comprobar su estado.

## FUNCIONALIDADES AVANZADAS

- **RandomizedDelaySec:** Se usa para evitar que, al encender el PC o un servidor, todos los timers pendientes no se ejecuten exactamente al mismo segundo sino que de forma aleatoria en un intervalo de tiempo, con el objetivo de no saturar el procesador o el servidor.
- El monitoreo se realiza con **journalctl -u**, permitiendo ver fallos exactos en tiempo real. Para las dependencias, se usa **After=** para definir el orden y **Wants=** para vincular el timer a requisitos como la red.
- Un **temporizador monótono** que cuenta el tiempo transcurrido desde un evento (*OnBootSec*), mientras que un **reloj de pared** para fechas y horas específicas (*OnCalendar*). El monótono es lineal y no se ve afectado por cambios en la hora del sistema, a diferencia del de pared.